WDD 131

JavaScript
Study Guide

Dynamic Web Fundamentals | Weeks 1-8

Covers: Data Types • Objects • DOM • Accessibility • Loops • Array Methods

Focus: Sections 5 (Trace the code) and 6 (Array methods) are worth the most points!

SECTION 1 DATA TYPES

Every value in JavaScript has a type. The exam will ask you to identify the type of a given value. Memorize this table.

Type	Example	How to recognize it
string	`"hello" or 'hello'`	Has quotation marks around it
number	`42 or 3.14`	Just a number — no quotes
boolean	`true or false`	Only ever true or false
null	`null`	The word null by itself
undefined	`undefined`	Variable declared but never given a value
array	`["a", "b", "c"]`	Square brackets [ ]
object	`{ x: 1, y: 2 }`	Curly braces { } with key: value pairs
function	`function greet() {}`	The word function followed by ( )

EXAM	On your previous exam, you got all data type questions correct! Keep it up.

Variables: let vs const

You declare variables using let or const. You used both throughout all 8 weeks of assignments.

```
const PI = 3.14;          // CONSTANT — cannot be changed later
let radius = 3;           // let — can be reassigned

radius = 5;               // OK — let allows reassignment
PI = 3.14159;             // ERROR — const cannot be changed!
```

TIP	Rule: Use const by default. Use let when you know the value will change (like a counter in a loop).

Scope — where can variables be used?

Scope means: where in the code can you access a variable? Variables declared with let or const inside curly braces { } only exist inside those braces.

```
let course = "CSE131";    // GLOBAL — works anywhere in the file
```

```
if (true) {
  let student = "John"; // BLOCK scope — only inside this { }
  console.log(course);  // OK — course is global
  console.log(student); // OK — inside the same block
}

console.log(course);    // OK — course is global
console.log(student);   // ERROR — student is gone outside the block
```

TIP	Memory trick: curly braces { } are like a room. Variables declared inside can't leave the room. Variables declared outside can go anywhere.

SECTION 2 OBJECTS

An object groups related data together using key: value pairs inside curly braces. You used objects in almost every assignment — movies, students, books, characters, courses, hikes, and recipes.

Creating an object

```
const product = {            // label A on exam
  name: "Laptop",            // label B — string value
  price: 799,                // label C — number value
  inStock: true,             // label D — boolean value
  specs: { ram: 16, storage: 512 }  // label E — nested object
};
```

Accessing object properties

There are two ways to get a value out of an object — dot notation and bracket notation. They do exactly the same thing.

Dot notation (most common)	Bracket notation
`product.name` // "Laptop"	`product["name"]` // "Laptop"
`product.price` // 799	`product["price"]` // 799
`product.specs.ram` // 16	

```
// Accessing nested objects: go one dot at a time
product.specs.ram     // first get specs, then get ram inside it
// result: 16
```

EXAM	Your exam: Q7 = number (label C). Q8 = object (label E). Q9 = product.specs.ram. Q10 = D (boolean). Q11 = 799.

Objects with methods (functions inside objects)

A method is just a function stored inside an object. Your Week 8 character card and courses assignments both used this pattern.

```
const character = {
  name: "Snortleblat",
  health: 100,
  level: 5,
```

```
  attacked: function() {          // this is a METHOD
    this.health -= 20;            // 'this' means 'this object'
  },
  levelUp: function() {
    this.level += 1;
  },
  reset: function() {
    this.health = 100;
    this.level = 5;
  }
};

character.attacked();             // call the method
console.log(character.health);    // 80
```

TIP	'this' inside a method always refers to the object it belongs to. So this.health is the same as character.health when called from inside the character object.

Arrays of objects

Almost every assignment from Week 6 onwards stored data as an array of objects. This is the most common pattern in web development.

```
const students = [
  { last: 'Andrus', first: 'Aaron', grade: 'A' },
  { last: 'Masa',   first: 'Manny', grade: 'C' },
  { last: 'Tanda',  first: 'Tamanda', grade: 'B' }
];

students[0].first    // 'Aaron'   (index 0 = first item)
students[1].grade    // 'C'       (index 1 = second item)
students[2].last     // 'Tanda'   (index 2 = third item)
```

SECTION 3 THE DOM

The DOM (Document Object Model) is how JavaScript reads and changes HTML on the page. Every HTML element is accessible as a JavaScript object.

The 4 selector methods — and their key differences

Method	Returns	Used in your assignments
`querySelector('#id')`	First match only	Every assignment with JS
`querySelector('.class')`	First match only	Week 2 theme switcher
`querySelectorAll('.class')`	All matches (NodeList)	Star ratings (weeks 5–6)
`getElementById('id')`	Single element by id	Week 6 movie list

EXAM	Exam section 3 key answers: querySelector('.nav-item') = 1 result. querySelectorAll('.nav-item') = 3. querySelectorAll('.nav-item')[1] = label D (index 1 = second item).

Changing content and styles with JS

```
const elem = document.querySelector('h1');

elem.textContent = "New title";            // change the text
elem.innerHTML = "<strong>Bold</strong>"; // change the HTML inside
elem.style.color = "red";                  // change CSS color
elem.style.backgroundColor = "blue";      // camelCase for CSS properties
elem.src = "new-image.jpg";               // change an image src
```

classList — adding and removing CSS classes

Used in your hamburger menu (Weeks 3-4) and modal (Week 4). classList lets JavaScript turn CSS classes on and off.

```
const nav = document.querySelector('nav');

nav.classList.add('show');        // adds the class 'show'
nav.classList.remove('show');     // removes the class 'show'
nav.classList.toggle('show');     // adds if missing, removes if present
nav.classList.contains('show')   // returns true or false
```

Creating new elements and adding them to the page

Used in Weeks 6-8 to build movie cards, book grids, and student lists dynamically from data arrays.

```
// 1. Create the element
const li = document.createElement('li');

// 2. Set its content
li.textContent = 'Aaron Andrus';

// 3. Add it to an existing element on the page
document.querySelector('ul').appendChild(li);
```

Event listeners

```
const btn = document.querySelector('button');

btn.addEventListener('click', function() {
  console.log('Button was clicked!');
});

// Common events used in your assignments:
// 'click'    — button or element clicked
// 'change'   — select dropdown changed (Weeks 2, 7)
// 'submit'   — form submitted (Week 7)
// 'resize'   — browser window resized (Week 4 hamburger)
// 'keypress' — keyboard key pressed (Week 8 hikes search)
```

TIP	The second argument to addEventListener is always a function. It runs automatically when the event happens — you never call it yourself.

SECTION 4 ACCESSIBILITY & ARIA

ARIA (Accessible Rich Internet Applications) attributes help screen readers understand your page. These appeared in almost every assignment from Week 5 onwards.

The four most important ARIA attributes

Attribute	What it does
`aria-label="..."`	Gives an element a readable name. Used for star ratings and icon-only buttons.
`aria-hidden="true"`	Hides the element from screen readers completely — they skip over it.
`aria-expanded="true/false"`	For dropdowns and menus. Says whether the panel is currently open (true) or closed (false).
`role="img"`	Tells the screen reader: treat this element like an image. Used for emoji star ratings.

Star rating pattern — used in every assignment from Week 5 on

```
// The problem: screen readers read '★★★★' as 'star star star star'
// The solution: use aria-label to explain what it means

<span role="img" aria-label="Rating: 4 out of 5 stars">
  <span aria-hidden="true">★★★★☆</span>
</span>

// Setting aria-label with JavaScript (from your Week 5-6 assignments):
const rating = elem.innerText.length;  // count how many star characters
elem.setAttribute('aria-label', `${rating} out of 5 stars`);
```

Exam review — section 4 answers

```
// Q18: star div — screen reader has no context
// ANSWER: aria-label="4 out of 5 stars"  (you missed the value)

// Q19: nav hidden with CSS, also hide from screen readers?
// ANSWER: aria-hidden="true"

// Q20: button with only an icon, no visible text
// ANSWER: aria-label="Close menu" (gives it an accessible name)

// Q21: menu is currently CLOSED
// ANSWER: aria-expanded="false"  (you got this right!)
```

TIP	Memory trick: aria-label = 'say this out loud.' aria-hidden = 'be silent about this.' aria-expanded = 'am I open or closed?'

SECTION 5 TRACE THE CODE (HIGH PRIORITY)

This section asks you to mentally run code and write the final value of a variable. You need to slow down and trace every step.

if / else if / else — the basics

```
if (condition) {
  // runs ONLY if condition is TRUE
} else if (anotherCondition) {
  // runs ONLY if first was false AND this is true
} else {
  // runs if NONE of the above conditions were true
}
```

Comparison operators — used in every condition

Operator	Meaning	Example
`===`	strictly equal	`5 === 5 is true.  5 === '5' is false`
`!==`	not equal	`5 !== 3 is true`
`>`	greater than	`10 > 5 is true`
`<`	less than	`3 < 10 is true`
`>=`	greater than or equal	`18 >= 18 is true  (includes equal!)`
`<=`	less than or equal	`5 <= 10 is true`

for loops — running code multiple times

A for loop has three parts. Understanding each part is key to tracing loops correctly.

```
for (let i = 0; i < 3; i++) {
//   PART 1    PART 2   PART 3
//
// PART 1: let i = 0   — starting value (runs once at the beginning)
// PART 2: i < 3       — keep going WHILE this is true (checked each loop)
// PART 3: i++         — add 1 to i after each loop  (i++ means i = i + 1)
}

// This loop runs with i = 0, then i = 1, then i = 2.
// When i = 3, the condition i < 3 is false, so it stops.
```

Tracing examples from your exam

Example 1 (Q22) — score:

```
let score = 15;
if (score > 20) {     // Is 15 > 20? NO — skip this block
  score = score + 10;
} else {              // Go to else
  score = score - 5;  // 15 - 5 = 10
}
// Final value of score: 10  (you got this right!)
```

Example 2 (Q23) — msg:

```
let count = 0;
let msg = "";
if (count === 0) {    // Is 0 === 0? YES
  msg = "empty";    // runs this
} else {
  msg = "has items";  // skipped
}
// Final value of msg: "empty"  (you got this right!)
```

Example 3 (Q24) — total (loop tracing):

```
let total = 0;
for (let i = 1; i <= 4; i++) {
  total = total + i;
}

// Trace each step:
// i=1:  total = 0 + 1 = 1
// i=2:  total = 1 + 2 = 3
// i=3:  total = 3 + 3 = 6
// i=4:  total = 6 + 4 = 10
// i=5:  5 <= 4 is FALSE — loop stops
// Final value of total: 10  (you got this right!)
```

Example 4 (Q25) — x (loop tracing):

```
let x = 10;
for (let i = 0; i < 3; i++) {
  x = x - i;
}

// Trace each step:
```

```
// i=0:  x = 10 - 0 = 10
// i=1:  x = 10 - 1 = 9
// i=2:  x = 9  - 2 = 7
// i=3:  3 < 3 is FALSE — loop stops
// Final value of x: 7  (your answer was correct — 7!)
```

EXAM	Your biggest loop tracing mistake: write out EVERY step on paper. Don't try to do it in your head. The errors happen when you skip steps.

Template literals — used everywhere in your code

Template literals use backtick characters (`) and allow you to embed variables directly inside a string using ${}.

```
let name = "Aaron";
let grade = "A";

// Old way (error-prone):
"Hello " + name + ", your grade is " + grade

// Template literal (clean and easy):
`Hello ${name}, your grade is ${grade}`
// Result: "Hello Aaron, your grade is A"

// You used this constantly in assignments:
`${student.first} ${student.last} - Grade: ${student.grade}`
`Class GPA: ${gpa.toFixed(2)}`
`Course Name: ${aCourse.name}`
```

TIP	Backticks look like this: ` — found on the key to the left of the number 1 on your keyboard. NOT a single quote. NOT a double quote.

SECTION 6 ARRAY METHODS (HIGH PRIORITY)

This is the most important section. You lost points here on your exam because of a close read on filter. Master each method below.

Arrow functions — used in all array methods

Arrow functions are a shorter way to write functions. All the array methods use them.

```
// Regular function:
function double(n) { return n * 2; }

// Arrow function — shorter, same result:
const double = (n) => { return n * 2; }

// Even shorter (single expression — no curly braces needed):
const double = n => n * 2;

// All three work identically. Your assignments use all three styles.
```

forEach — do something for each item

```
const nums = [1, 2, 3];

nums.forEach((n) => {
  console.log(n * 3);
});

// Runs: console.log(1*3) → 3
//       console.log(2*3) → 6
//       console.log(3*3) → 9

// Q26: What is the LAST value printed? → 9  (you got this right!)

// IMPORTANT: forEach returns nothing. It just performs an action.
```

filter — keep only items that pass a test

```
const ages = [14, 22, 17, 30, 19];

const adults = ages.filter((age) => {
  return age >= 18;   // keep the item if this is true
});
```

```
// Check each item:
// 14 >= 18? NO  — removed
// 22 >= 18? YES — kept
// 17 >= 18? NO  — removed
// 30 >= 18? YES — kept
// 19 >= 18? YES — kept  <-- YOU MISSED THIS ONE!

// adults = [22, 30, 19]   (you wrote [22, 30] — forgot 19!)
```

NOTE	CRITICAL: >= means 'greater than OR equal to'. 19 >= 18 is TRUE because 19 is greater than 18. Always check every single item!

map — transform every item into something new

```
const prices = [10, 20, 30];

const doubled = prices.map((p) => {
  return p * 2;
});

// doubled = [20, 40, 60]   (you got this right — Q28!)

// CRITICAL: map does NOT change the original array!
// prices is still [10, 20, 30] after the map.
// Q29: Does .map() change the original array? → NO  (you got this!)
```

reduce — combine all items into a single value

```
const nums = [2, 4, 6, 8];

const total = nums.reduce((acc, n) => {
  return acc + n;
}, 0);  // <-- the 0 is the STARTING VALUE of acc

// Q30: Starting value of acc = 0  (you got this right!)

// Trace the steps:
// acc=0,  n=2 → new acc = 0+2  = 2
// acc=2,  n=4 → new acc = 2+4  = 6
// acc=6,  n=6 → new acc = 6+6  = 12
// acc=12, n=8 → new acc = 12+8 = 20

// Q31: Value of total = 20  (you got this right!)
```

Chaining methods — filter then map

You can chain array methods. The output of the first method becomes the input of the next. Read left to right, top to bottom.

```
const items = [5, 3, 8, 1, 9, 2];

const result = items
  .filter((n) => { return n > 4; })  // STEP 1: keep only [5, 8, 9]
  .map((n) => { return n * 10; });   // STEP 2: multiply each → [50, 80,
90]

// Q32: Value of result = [50, 80, 90]  (you got this right!)
```

Quick reference — all array methods at a glance

Method	Returns	Changes original?	Use when...
`forEach`	nothing (undefined)	No	Do something with each item
`filter`	new array (subset)	No	Keep only items that match a condition
`map`	new array (same length)	No	Transform every item into something new
`reduce`	single value	No	Add up / combine all items into one thing
`find`	one item (or undefined)	No	Get the first item that matches
`findIndex`	one index number	No	Get the position of the first match
`sort`	same array (reordered)	YES!	Reorder the items
`includes`	true or false	No	Check if an item exists in the array

TIP	The safest question to ask: 'What does this method RETURN?' forEach = nothing. filter/map = new array. reduce = single value.

filter with arrays of objects (from your assignments)

```
// From your hikes assignment (Week 8):
const hikes = [
  { name: 'Bechler Falls', difficulty: 1, tags: ['Waterfall',
'Yellowstone'] },
  { name: 'Menan Butte',   difficulty: 2, tags: ['Volcanic', 'View'] },
  { name: 'Denanda Falls', difficulty: 3, tags: ['Waterfall', 'Caves'] },
```

```
];

// Filter to only easy hikes (difficulty 1):
const easyHikes = hikes.filter(hike => hike.difficulty === 1);
// easyHikes = [{ name: 'Bechler Falls', ... }]

// Filter by tag:
const waterfallHikes = hikes.filter(hike =>
hike.tags.includes('Waterfall'));
// result includes Bechler Falls and Denanda Falls
```

SECTION + ADDITIONAL CONCEPTS FROM YOUR ASSIGNMENTS

switch statements (Week 6 — student grades)

A switch is like an if/else if chain but cleaner when you have many exact-match cases. Always include break at the end of each case.

```
function convert(grade) {
  let points;
  switch (grade) {
    case 'A': points = 4; break;  // break stops it from falling through
    case 'B': points = 3; break;
    case 'C': points = 2; break;
    case 'D': points = 1; break;
    case 'F': points = 0; break;
    default:  alert('not a valid grade'); return 0;  // catches anything
else
  }
  return points;
}

convert('A');  // returns 4
convert('C');  // returns 2
```

TIP	Forgetting break causes 'fall-through' — the code runs all subsequent cases too. Always include break unless you intentionally want fall-through.

map with template literals (Week 6-8 — movies, books, hikes)

The most common pattern in your assignments: loop over an array of objects, generate HTML for each, and insert it into the page.

```
const movies = [
  { title: 'Spider-Man', stars: '⭐⭐⭐⭐⭐' },
  { title: 'Luca',       stars: '⭐⭐⭐⭐' },
];

const moviesList = document.getElementById('movie-list');

movies.forEach((movie) => {
  const article = document.createElement('article');
  article.innerHTML = `
    <h2>${movie.title}</h2>
    <p>Rating: ${movie.stars}</p>
  `;
  moviesList.appendChild(article);
```

```
});
```

form validation (Week 7 — checkout form)

```
const form = document.querySelector('#checkoutForm');

form.addEventListener('submit', function(event) {
  event.preventDefault();  // STOP the form from refreshing the page
                           // Now you can validate the inputs yourself
  let errorMsg = '';

  const cardNum =
document.querySelector('#creditCardNumber').value.trim();

  if (!/^\d{16}$/.test(cardNum)) {           // regex: exactly 16 digits
    errorMsg += 'Card number must be 16 digits\n';
  }

  if (errorMsg !== '') {
    document.querySelector('.errors').textContent = errorMsg;
    return;  // stop — don't submit
  }

  form.innerHTML = '<h2>Thank you!</h2>';  // success!
});
```

sort — reordering an array

Used in your hikes assignment to sort by difficulty, and in the recipe book to sort alphabetically.

```
// Sort numbers with a compare function:
function compareHikes(a, b) {
  if (a.difficulty < b.difficulty) { return -1; }  // a comes first
  if (a.difficulty > b.difficulty) { return 1;  }  // b comes first
  return 0;  // same — keep order
}
filteredHikes.sort(compareHikes);  // now sorted easiest to hardest

// Sort alphabetically (from your recipe book):
filtered.sort((a, b) => a.name.localeCompare(b.name));
```

The defer attribute on script tags

```
// In your HTML <head>:
```

```
<script src="app.js" defer></script>

// Without defer: JS runs BEFORE HTML is loaded → querySelector returns
null!
// With defer:    JS waits until ALL HTML is loaded → querySelector works
fine

// All your assignments from Week 2 onwards used defer.
```

Responsive design with media queries (Weeks 3-4)

```
/* Mobile-first: default styles for small screens */
#gallery {
  grid-template-columns: 1fr;  /* 1 column on mobile */
}

/* Tablet: 768px and wider */
@media screen and (min-width: 768px) {
  #gallery {
    grid-template-columns: 1fr 1fr;  /* 2 columns */
  }
}

/* Desktop: 1100px and wider */
@media screen and (min-width: 1100px) {
  #gallery {
    grid-template-columns: 1fr 1fr 1fr;  /* 3 columns */
  }
}
```

SECTION P PRACTICE PROBLEMS

Work through these by hand. Write your answer before looking at the solution. These are the kinds of problems that appear on your exam.

Trace the code

Problem 1

```
let x = 8;
if (x >= 10) {
  x = x + 5;
} else if (x >= 5) {
  x = x + 2;
} else {
  x = x - 1;
}
// Final value of x: ____
// ANSWER: 10  (8 >= 5 is true, so 8+2=10)
```

Problem 2

```
let result = 0;
for (let i = 1; i <= 5; i++) {
  if (i % 2 === 0) {     // % is modulo — remainder after division
    result = result + i; // only add EVEN numbers
  }
}
// Final value of result: ____
// ANSWER: 6  (i=2 adds 2, i=4 adds 4, total = 6)
```

Array methods

Problem 3

```
const scores = [85, 92, 47, 63, 78, 91];

const passing = scores.filter(score => score >= 70);
// What does passing contain? ____
// ANSWER: [85, 92, 78, 91]  (47 and 63 fail the >= 70 test)
```

Problem 4

```
const words = ['apple', 'banana', 'cherry'];
const lengths = words.map(word => word.length);
// What does lengths contain? ____
// ANSWER: [5, 6, 6]  ('apple'=5, 'banana'=6, 'cherry'=6)
```

Problem 5

```
const nums = [1, 2, 3, 4, 5];
const sum = nums.reduce((acc, n) => acc + n, 0);
// Starting value of acc: ____
// Final value of sum: ____
// ANSWER: acc starts at 0. Sum = 1+2+3+4+5 = 15
```

Problem 6 — chained methods

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];

const result = numbers
  .filter(n => n % 2 === 0)    // Step 1: keep only even numbers
  .map(n => n * n);            // Step 2: square each

// What does result contain? ____
// STEP 1 result: [2, 4, 6, 8]
// STEP 2 result: [4, 16, 36, 64]
// ANSWER: [4, 16, 36, 64]
```

Objects

Problem 7

```
const movie = {
  title: 'Luca',
  year: 2021,
  rating: 4.5,
  studio: { name: 'Pixar', country: 'USA' }
};

// What is the data type of movie.year? ____
// ANSWER: number

// Write dot notation to get the country:
// ANSWER: movie.studio.country

// What does movie['title'] return?
// ANSWER: 'Luca'
```